```
git clone https://github.com/<your-account>/private-rag-agent
cd private-rag-agent
pip install -r requirements.txt
```

Private RAG Agent — A Fully Offline, Local Private Multi-Agent RAG Assistant

> 2026 AMD AI DevMaster Hackathon · Track 2 (Development & Local Deployment of Private AI Agents)
>
> Fully offline · Local inference · Data never leaves the machine · AMD Radeon GPU / ROCm optimized

---

Overview

Private RAG Agent is a fully offline, locally hosted, private multi-agent RAG system. Import your private documents, ask complex questions, and a collaborative pipeline of agents — Decompose, parallel Research, Fact-Check, Synthesize — produces sourced, verifiable answers. No cloud API, no telemetry, no third-party inference: chunking, embedding, retrieval, and inference all happen on your own machine.

Core selling points

- Data never leaves the machine — parsing, embedding, vector storage, retrieval, and inference are all local; nothing is uploaded anywhere.
- AMD GPU local inference — multi-backend support (llama.cpp HIP / Ollama ROCm / vLLM), including first-class support for AMD Radeon Cloud.
- Multi-agent collaboration — a Decomposer, parallel Researchers, a Fact-Checker, and a Synthesizer mirror industrial multi-agent RAG architectures.
- Traceable and auditable — hybrid retrieval, cross-encoder reranking, per-sentence groundedness checks, and clickable citations that jump straight to the source text.
- Multi-format documents — PDF, Word, Markdown, Excel, and PowerPoint are all supported.
- Professional dark UI — a custom FastAPI + SSE + native HTML/CSS/JS front end with a live activity timeline, per-sentence verification panel, and a real-time AMD GPU monitoring panel.

Quick Start

Prerequisites

- AMD Radeon GPU (recommended: RX 7900 XTX / W7900, 16 GB+ VRAM, or Radeon Cloud)
- ROCm 6.x (Linux) or Ollama (Windows)
- Python 3.10+

1. Install

```
ollama pull qwen3:8b        # LLM
ollama pull bge-m3          # Embedding model
ollama serve
```

2. Start a local inference backend

Path A — Ollama (simplest, works on Windows and Linux):

```
CMAKE_ARGS="-DGGML_HIP=ON" pip install llama-cpp-python
# point config.yaml model.name at your GGUF file (see below)
```

Path B — llama.cpp + ROCm (native AMD GPU):

```
python main.py ingest data/docs/   # batch-import the docs directory
python main.py ui                  # open http://localhost:7860
```

Path C — vLLM + ROCm (high concurrency / Radeon Cloud):

Use the `rocm/vllm` Docker image or a Radeon Cloud workspace, then set `model.backend: vllm` and `model.base_url: http://localhost:8000/v1`. The included `deploy_rc.sh` automates the whole Radeon Cloud flow.

3. Ingest documents and launch the web UI

```
docker compose up -d --build
docker exec -it private-rag-ollama ollama pull qwen3:8b
docker exec -it private-rag-ollama ollama pull bge-m3
```

Other CLI entry points: `python main.py ask "question"` (CLI Q&A) and `python main.py bench` (AMD GPU benchmark).

Docker (one-command containerized deployment)

```
┌──────────────────────────────────────────────────────────┐
│ UI (FastAPI + SSE + native HTML/CSS/JS)                  │
│  ├── streaming chat (typewriter effect + activity timeline)  │
│  ├── run-trajectory panel (live tool / verification events)  │
│  ├── GPU monitoring panel (utilization / VRAM / temp / clock) │
│  └── source deep-dive drawer (click citation → source text)  │
├──────────────────────────────────────────────────────────┤
│ Agent layer                                              │
│  ├── Agent loop    perceive → plan → tools → answer → verify  │
│  ├── MultiAgent    decompose → parallel research → check →   │
│  │                 synthesize                            │
│  ├── Planner       task planning (only complex questions)    │
│  ├── Memory        multi-turn + long-term memory             │
│  └── Tools         rag_search / read_doc / list_docs /       │
│                    summarize / web_search_offline            │
├──────────────────────────────────────────────────────────┤
│ RAG retrieval pipeline (hybrid retrieval + reranking)    │
│  ├── parser.py         PDF/Word/MD/Excel/PPT parsing     │
│  ├── chunk_text        smart chunking (512 chars, 64 overlap) │
│  ├── query rewriting   LLM generates search-term variants  │
│  ├── vector search     ChromaDB (bge-m3, cosine)         │
│  ├── BM25              keyword recall (proper nouns/abbrev.)  │
│  ├── RRF fusion        Reciprocal Rank Fusion            │
│  └── cross-encoder     bge-reranker-v2-m3 rerank (Top-k) │
├──────────────────────────────────────────────────────────┤
│ Trustworthiness verification                             │
│  ├── groundedness      3-gram overlap: answer sentences vs.  │
│  │                     retrieved source text             │
│  ├── citation          cited filenames must really exist │
│  └── LLM-as-judge      claim-by-claim fact checking      │
├──────────────────────────────────────────────────────────┤
│ Inference backends (llm.py)                              │
│  ├── ollama / llama.cpp (GGUF + ROCm)                    │
│  └── vLLM (Radeon Cloud / ROCm, high concurrency)        │
└──────────────────────────────────────────────────────────┘
```

On an AMD + ROCm Linux host, uncomment the `devices` block under the `ollama` service in `docker-compose.yml` to pass the GPU into the container. The vector store is ChromaDB persisted under `./data`, so documents never leave the host.

Architecture

End-to-end pipeline

```
User question
   │
   ▼
1. Decomposer ── splits into n mutually independent sub-questions
   │
   ▼
2. Researchers (n parallel sub-agents) ── each retrieves from the
   │  knowledge base and produces a sub-report
   │  (each sub-agent streams tool/retrieval events to the UI
   │   activity panel in real time)
   ▼
3. Fact-Checker ── two-level verification on every sub-report
   │    · groundedness: per-sentence 3-gram overlap vs. retrieved text
   │    · LLM-as-judge: claim-by-claim verdict
   ▼
4. Synthesizer ── composes the final answer ONLY from verified content
   │
   ▼
5. Final verification ── groundedness + citation checks re-run on the
      complete final answer
```

The multi-agent pipeline (the "rigor" story)

```
query rewriting (LLM, optional)
   → vector search (ChromaDB, semantic)  +  BM25 (keyword)
   → score fusion (Reciprocal Rank Fusion)
   → cross-encoder rerank (bge-reranker-v2-m3, optional)
   → ranked chunks with source + score
```

Why parallelism (an AMD-flavored motivation): on a local GPU, multiple inference requests queue back-to-back, so converting a serial chain into parallel execution sharply reduces the wall-clock time of multi-step research tasks. All n sub-agents share a single copy of the embedding / reranker models (module-level singletons), so VRAM footprint increases by only one copy regardless of n.

RAG Retrieval Pipeline

The retrieval stack follows the industry-standard hybrid recipe, entirely local and offline:

```
rocminfo                       # confirm ROCm device
rocm-smi                       # monitor VRAM / temperature / utilization
ollama ps                      # confirm the model is 100% GPU-resident
python benchmarks/bench_amd.py # auto-detect platform and report tokens/s
```

- Query rewriting — one lightweight local LLM call generates up to 3 query variants (originals plus rewritten forms that fill in proper nouns and abbreviations), improving recall without extra cost.
- Vector search — ChromaDB with cosine space and `bge-m3` embeddings (1024 dimensions), capturing semantic similarity.
- BM25 — `rank_bm25` with a Chinese-aware tokenizer (2-gram sliding window for CJK, word tokens for English). This catches exact matches for proper nouns and abbreviations that pure vector search misses.
- RRF fusion — Reciprocal Rank Fusion merges the two ranked lists with configurable weights (BM25 0.4, vector 0.6).

- Cross-encoder rerank — `bge-reranker-v2-m3` jointly scores each (query, document) pair, which is the single highest-leverage precision improvement in RAG; the candidate pool is 16 chunks before reranking, cut to `top_k` (default 4) after.
- Chunking — 512 characters with 64-character overlap, applied by the multi-format parser (PDF / DOCX / MD / XLSX / PPTX / TXT / CSV / JSON).
- Graceful degradation — any stage can fail or be disabled without breaking the pipeline; pure vector search is the fallback floor.

Trustworthiness: Groundedness + Citation Verification

This is the project's differentiator and the part judges can see working. The system refuses to silently hallucinate: every answer sentence is checked against the text that was actually retrieved, and every citation is checked against the documents that actually exist in the knowledge base.

Groundedness (deterministic, per sentence)

`agent/verify.py` implements a fully deterministic check — fast, stable, and no extra LLM calls:

- Normalize both answer and retrieved source text (strip punctuation, whitespace, markdown, emoji).
- Split the answer into sentences/list items.
- Compute the character 3-gram overlap of each sentence against the union of retrieved source text.
- Label each sentence supported (overlap > 0.30), partial (> 0.12), or unsupported, and derive an overall grounding score (0–1).

Chinese is not segmented, so character n-grams are robust to paraphrase; the 3-gram window resists minor rewording while still catching fabrication.

Citation verification

A regex (`[ :file]`, `[file.md]`, etc.) extracts every citation from the answer, then each one is checked for existence against the set of documents actually retrieved in this session. The result is a citation precision score (0–1) plus a per-citation pass/fail list. Fabricated filenames are caught and surfaced.

LLM-as-judge (claim level, multi-agent mode)

Each sub-report is additionally passed to an LLM judge that emits a JSON verdict per factual claim (`{claim, supported, reason}`). Unsupported claims are reported back with reasons and carried into the final answer as flagged limitations.

Trust score in the UI

The final answer ships with a verification object — grounding score, per-sentence panel, and per-citation checks — rendered by the UI as a trust-score panel. Clicking a citation opens the source drawer showing the original document text with the query terms highlighted. This is a closed audit loop: every claim can be traced to a real chunk in the knowledge base.

AMD / ROCm Optimization (40% of Track 2 scoring)

1. Multi-backend inference

Backend	Use case	Notes	
llama.cpp (HIP)	Single machine, single GPU	`CMAKE_ARGS="-DGGML_HIP=ON"` ; GGUF with full GPU offload (`n_gpu_layers=-1`)	
Ollama (ROCm)	Out of the box	Auto ROCm backend on Linux; `ollama ps` confirms 100% GPU residency	
vLLM (ROCm)	High concurrency / Radeon Cloud	`rocm/vllm` Docker image; OpenAI-compatible API built in — `deploy_rc.sh` wires it up	
Quantization	VRAM	Speed	Recommended
Q8_0	~8.2 GB	Fast	Maximum fidelity
Q4_K_M	~4.6 GB	Fastest	Default balanced choice
llama.cpp (HIP)	Single machine, single GPU	`CMAKE_ARGS="-DGGML_HIP=ON"` ; GGUF with full GPU offload (`n_gpu_layers=-1`)	
Ollama (ROCm)	Out of the box	Auto ROCm backend on Linux; `ollama ps` confirms 100% GPU residency	
vLLM (ROCm)	High concurrency / Radeon Cloud	`rocm/vllm` Docker image; OpenAI-compatible API built in — `deploy_rc.sh` wires it up	
Quantization	VRAM	Speed	Recommended
Q8_0	~8.2 GB	Fast	Maximum fidelity
Q4_K_M	~4.6 GB	Fastest	Default balanced choice
llama.cpp (HIP)	Single machine, single GPU	`CMAKE_ARGS="-DGGML_HIP=ON"` ; GGUF with full GPU offload (`n_gpu_layers=-1`)	
Ollama (ROCm)	Out of the box	Auto ROCm backend on Linux; `ollama ps` confirms 100% GPU residency	

llama.cpp (HIP)	Single machine, single GPU	`CMAKE_ARGS="-DGGML_HIP=ON"` ; GGUF with full GPU offload (`n_gpu_layers=-1`)	
vLLM (ROCm)	High concurrency / Radeon Cloud	`rocm/vllm` Docker image; OpenAI-compatible API built in — `deploy_rc.sh` wires it up	
Quantization	VRAM	Speed	Recommended
Q8_0	~8.2 GB	Fast	Maximum fidelity
Q4_K_M	~4.6 GB	Fastest	Default balanced choice
Ollama (ROCm)	Out of the box	Auto ROCm backend on Linux; `ollama ps` confirms 100% GPU residency	
vLLM (ROCm)	High concurrency / Radeon Cloud	`rocm/vllm` Docker image; OpenAI-compatible API built in — `deploy_rc.sh` wires it up	
Quantization	VRAM	Speed	Recommended
Q8_0	~8.2 GB	Fast	Maximum fidelity
Q4_K_M	~4.6 GB	Fastest	Default balanced choice
vLLM (ROCm)	High concurrency / Radeon Cloud	`rocm/vllm` Docker image; OpenAI-compatible API built in — `deploy_rc.sh` wires it up	
Quantization	VRAM	Speed	Recommended
Q8_0	~8.2 GB	Fast	Maximum fidelity
Q4_K_M	~4.6 GB	Fastest	Default balanced choice

2. Quantization (GGUF)

Quantization	VRAM	Speed	Recommended
Q8_0	~8.2 GB	Fast	Maximum fidelity
Q4_K_M	~4.6 GB	Fastest	Default balanced choice
Q8_0	~8.2 GB	Fast	Maximum fidelity
Q4_K_M	~4.6 GB	Fastest	Default balanced choice

Q8_0	~8.2 GB	Fast	Maximum fidelity
Q4_K_M	~4.6 GB	Fastest	Default balanced choice
Q4_K_M	~4.6 GB	Fastest	Default balanced choice

3. GPU resource throttling (shared small models)

- Embedding and reranker models are module-level singletons — across n parallel researcher agents, exactly one copy is loaded into VRAM, no matter how many agents run.
- Retrieval and reranking load lazily on first use, so the app starts without pre-committing VRAM.
- All access to native ChromaDB (hnswlib) and torch cross-encoder resources is serialized through a process-level reentrant lock. hnswlib and torch native code are not thread-safe; concurrent `collection.query` or cross-encoder inference can segfault (exit 139). Serializing only the native paths adds milliseconds of contention while LLM inference — the expensive part — runs concurrently through the Ollama/vLLM service.
- The UI's GPU panel reads real metrics via `pyamdgpuinfo`, falling back to `rocm-smi`, and to live demo data when no AMD GPU is present (so the interface stays functional during development).

4. Verification and monitoring

```
private-rag-agent/
├── agent/
│   ├── agent.py        # Agent loop (planning / tools / memory / citation check)
│   ├── multi_agent.py  # Multi-agent orchestration (decompose → parallel
│   │                   # research → fact-check → synthesize)
│   ├── rag.py          # Hybrid retrieval (vector   BM25 → RRF → rerank)
│   ├── verify.py       # groundedness + citation verification
│   ├── llm.py          # Inference layer (ollama / llama.cpp / vLLM)
│   ├── parser.py       # Multi-format document parsing + smart chunking
│   ├── tools.py        # Tool registry (5 tools)
│   ├── memory.py       # Short-term + long-term memory
│   └── planner.py      # Task planning
├── ui/
│   ├── server.py       # FastAPI + SSE streaming API
│   └── static/         # Native HTML/CSS/JS (dark precision-instrument UI)
├── benchmarks/         # AMD ROCm benchmark
├── data/
│   ├── docs/           # Knowledge base source documents
│   └── vector_db/      # ChromaDB persistent store (created on ingest)
├── config.yaml         # Configuration (retrieval / chunking / quantization)
├── main.py             # CLI entry point
├── deploy_rc.sh        # Radeon Cloud deployment script
├── docker-compose.yml  # One-command containerized deployment
└── requirements.txt
```

The benchmark (`benchmarks/bench_amd.py`) detects the environment (ROCm vs. NVIDIA vs. CPU), runs a warmup pass to exclude model-loading time, and reports average latency and throughput per prompt.

Measured on the dev machine (NVIDIA RTX 4070 Laptop, Ollama qwen3:8b): CPU inference ~1.8 token/s. On Radeon Cloud (RX 7900 / ROCm) with GGUF Q4_K_M and full-layer offload, throughput is projected at a 5–10x improvement; the final submission includes an AMD comparison table under `benchmarks/`.

Directory Structure

```
model:
  backend: ollama            # ollama | llama_cpp | vllm
  name: qwen3:8b             # local model name (AMD: Qwen2.5-7B-Instruct-Q4_K_M GGUF)
  temperature: 0.3
  max_tokens: 2048
  top_p: 0.9

embedding:
  backend: ollama            # ollama | sentence_transformers | vllm
  name: bge-m3               # multilingual embedding model (1024-dim)
  dim: 1024

rag:
  chunk_size: 512            # chunk length in characters
  chunk_overlap: 64          # overlap between adjacent chunks
  top_k: 4                   # chunks returned to the agent / user
  vector_store: chroma       # chroma | faiss | sqlite_vec
  db_dir: ./data/vector_db
  docs_dir: ./data/docs      # drop documents here to ingest
  allowed_ext: [.pdf, .docx, .md, .txt, .csv, .json, .pptx, .xlsx]
  hybrid: true               # BM25 keyword + vector semantic search
  bm25_weight: 0.4           # BM25 fusion weight (vector is 0.6)
  rerank: true               # cross-encoder reranking
  rerank_model: BAAI/bge-reranker-v2-m3   # multilingual reranker
  retrieval_top_k: 16        # candidate pool size before reranking
  query_expansion: true      # LLM query rewriting

memory:
  max_history: 10            # conversation turns kept in short-term memory
  long_term: ./data/memory_long.json

tools:
  max_steps: 6               # max tool-call steps per agent task
  enabled: [rag_search, read_doc, list_docs, summarize, web_search_offline]

server:
  host: 0.0.0.0
  port: 7860
```

Configuration (`config.yaml`)

Key notes:

- Backend switching is one line: `ollama` → `llama_cpp` (point `model.name` at a GGUF file) → `vllm` (set `base_url`). The embedding backend is configured independently, so a vLLM LLM can be paired with a CPU sentence-transformers embedding model, as in the Radeon Cloud deployment.
- `OLLAMA_HOST` is honored as an environment variable (Docker sets it to the `ollama` service name).
- The UI GPU panel runs in demo mode when `server.demo_gpu` is enabled and no AMD GPU is detected.

Deployment Targets

- Local: `pip install -r requirements.txt` + Ollama (Path A) or llama.cpp HIP (Path B).
- Docker: `docker compose up -d --build` (Ollama + app, data persisted in `./data`).
- AMD Radeon Cloud: run `bash deploy_rc.sh` in an RC workspace. It detects the GPU (`amd-smi`/`rocm-smi`), installs vLLM (ROCm) if missing, launches the inference service

(auto-detecting the pre-downloaded RC model, e.g. Qwen3.6-35B-A3B-AWQ-4bit), rewrites `config.yaml` to the vLLM + sentence-transformers backend, and smoke-tests inference. The vector database stays dimension-compatible with local development (bge-m3, 1024-dim), so a local store can be reused on the cloud.

Submission Info (Track 2)

- Track: Track 2 (Development & Local Deployment of Private AI Agents)
- Team: _____
- Demo video: _____
- Submission deadline: 2026-08-06 23:59 (Beijing time)

License & Disclaimer

This project is a hackathon submission. All model weights and document data are stored locally on the machine; no user data is collected or transmitted. Software is provided as-is without warranty.